

2303A510A1 - Bhanu

Assignment - 12.4

Batch - 14

Lab 12 - Algorithms with AI Assistance - Sorting, Searching, and Optimizing Algorithms

Task 1: Bubble Sort for Ranking Exam Scores

Prompt:

- Implement Bubble Sort in Python to sort a list of student scores.
- Insert inline comments explaining key operations such as comparisons, swaps, and iteration passes

Code:

```
def bubble_sort(scores):
    n = len(scores) # Get the number of scores in the list
    # Traverse through all elements in the list
    for i in range(n):
        # Initialize a flag to check if any swapping occurs
        swapped = False
        # Last i elements are already in place, no need to check them
        for j in range(0, n-i-1):
            # Compare adjacent elements
            if scores[j] > scores[j+1]:
                # If the element is greater than the next element, swap them
                scores[j], scores[j+1] = scores[j+1], scores[j]
                swapped = True # Set the flag to True if a swap occurred
        # If no swapping occurred, the list is already sorted
        if not swapped:
            break # Exit the loop early since the list is sorted
    return scores # Return the sorted list of scores

# Example usage
student_scores = [88, 75, 92, 85, 79]
sorted_scores = bubble_sort(student_scores)
print("Sorted student scores:", sorted_scores) # Output the sorted list of scores
```

Output:

```
Sorted student scores: [75, 79, 85, 88, 92]
```

Explanation:

- The bubble sort algorithm has a best-case time complexity of $O(n)$ when the list is already sorted, as it only requires one pass to verify that the list is in order.
- The average and worst-case time complexity of bubble sort is $O(n^2)$ because in the worst case (when the list is sorted in reverse order), it requires n passes and each pass involves n comparisons, leading to $n * n = n^2$ operations.

Task 2: Improving Sorting for Nearly Sorted Attendance Records

Bubble Sort Code:

```
def bubble_sort(arr):
    n = len(arr)
    # Traverse through all elements in the array
    for i in range(n):
        # Last i elements are already in place, no need to check them
        for j in range(0, n - i - 1):
            # Swap if the element found is greater than the next element
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

#example usage with a nearly sorted list
nearly_sorted_arr = [1, 9, 3, 8, 4, 6, 7, 5, 2]
sorted_arr = bubble_sort(nearly_sorted_arr)
print("Bubble Sort" )
print("Sorted array:", sorted_arr)
```

Review:

- For nearly sorted data with only a few misplaced elements, Insertion Sort is more efficient than Bubble Sort.
- Insertion Sort performs better for nearly sorted data because it works by building a sorted portion of the array one element at a time. When the data is nearly sorted, Insertion Sort can quickly place each element in its correct position with fewer comparisons and movements. In contrast, Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order, which can lead to unnecessary comparisons and movements even when the data is nearly sorted. Therefore, Insertion Sort is more efficient for this type of data.

Prompt:

- Write a Python program for Insertion Sort.
- Add inline comments explaining shifting and insertion.
- Use a nearly sorted example list.

Code:

```
def insertion_sort(arr):
    # Traverse through 1 to len(arr)
    for i in range(1, len(arr)):
        key = arr[i] # The current element to be positioned
        j = i - 1    # The index of the last sorted element
        # Shift elements of arr[0..i-1], that are greater than key,
        # to one position ahead of their current position
        while j >= 0 and key < arr[j]:
            arr[j + 1] = arr[j] # Shift the element one position to the right
            j -= 1
        # Insert the key at its correct position in the sorted subarray
        arr[j + 1] = key
    return arr

# Example usage with a nearly sorted list
nearly_sorted_arr = [1, 2, 3, 5, 4, 6, 7]
sorted_arr = insertion_sort(nearly_sorted_arr)
print("Sorted array:", sorted_arr)
```

Output:

```
Bubble Sort
```

```
Sorted array: [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
Insertion Sort
```

```
Sorted array: [1, 2, 3, 4, 5, 6, 7]
```

Explanation:

Time Complexity of Insertion Sort:

- Best Case: $O(n)$ when the input array is already sorted, as it only requires one pass through the array.
- Average Case: $O(n^2)$ when the input array is in random order, as each element may need to be compared with every other element.
- Worst Case: $O(n^2)$ when the input array is sorted in reverse order, as each element will need to be compared with every other element.

Performance Comparison with Bubble Sort:

- Bubble Sort has a time complexity of $O(n^2)$ in the average and worst cases, and $O(n)$ in the best case when the array is already sorted. However, Bubble Sort requires multiple passes through the array to ensure it is sorted, which can be inefficient for nearly sorted data.
- Insertion Sort, on the other hand, is more efficient for nearly sorted data because it can quickly place elements in their correct positions with fewer comparisons and movements. When the input is nearly sorted, Insertion Sort can achieve a time complexity close to $O(n)$, as it will only need to make a few comparisons and shifts to sort the array. This makes Insertion Sort a better choice than Bubble Sort for nearly sorted datasets.

Task 3: Searching Student Records in a Database

Linear Search:

Prompt:

Write a Python function for Linear Search. Add a proper docstring explaining parameters and return value. Use an example of unsorted student roll numbers. Explain time complexity.

Code:

```
def linear_search(arr, target):
    """
    Perform a linear search for the target value in the given array.
    Parameters:
    arr (list): A list of unsorted student roll numbers.
    target (int): The roll number to search for.
    Returns:
    int: The index of the target if found, otherwise -1.
    """
    for index in range(len(arr)):
        if arr[index] == target:
            return index
    return -1

roll_numbers = [23, 45, 12, 67, 34]
target_roll = 12
result = linear_search(roll_numbers, target_roll)
print(result) # Output: 2
```

Output:

2

Explanation:

The time complexity of linear search is $O(n)$, where n is the number of elements in the array. This is because in the worst case, the algorithm needs to check each element once to find the target or determine that it is not present in the array.

Binary search:

Prompt:

Write a Python function for Binary Search. Include docstring. Explain why the list must be sorted. Provide example input and output. Explain time complexity.

Code:

```
def binary_search(arr, target):  
    """  
    Perform binary search on a sorted list to find the index of the target  
    element.  
    Parameters:  
    arr (list): A sorted list of elements.  
    target: The element to search for in the list.  
    Returns:  
    int: The index of the target element if found, otherwise -1.  
    Note:  
    The list must be sorted for binary search to work correctly. This is  
    because binary search relies on the ability to eliminate half of the  
    remaining elements at each step based on comparisons. If the list is not  
    sorted, this elimination process would not be valid, and the search would  
    fail to find the target efficiently.  
    """  
    left, right = 0, len(arr) - 1  
    while left <= right:  
        mid = left + (right - left) // 2  
        # Check if the target is present at mid  
        if arr[mid] == target:  
            return mid
```

```
# If target is greater, ignore the left half
elif arr[mid] < target:
    left = mid + 1

# If target is smaller, ignore the right half
else:
    right = mid - 1

# Target was not found in the list
return -1

# Example input and output
sorted_list = [1, 2, 3, 4, 5, 6, 7, 8, 9]
target = 5
result = binary_search(sorted_list, target)
print(f"Target {target} found at index: {result}")
```

Output:

```
Target 5 found at index: 4
```

Explanation:

Time complexity of binary search is $O(\log n)$ because with each step, it halves the number of elements to be checked. This makes it much more efficient than linear search ($O(n)$) for large sorted lists.

Comparison:

Linear Search:

- **Time Complexity:** $O(n)$ - In the worst case, it may have to check all elements in the list.
- **Use Cases:** Linear search is suitable for small datasets or when the data is unsorted.
- **Applicability:** It can be used on both sorted and unsorted data, but it is inefficient for large datasets.

Binary Search:

- **Time Complexity:** $O(\log n)$ - It divides the search interval in half with each step, making it much faster than linear search for large datasets.
- **Use Cases:** Binary search is ideal for large datasets that are sorted. It is commonly used in applications like searching in databases, finding elements in sorted arrays, and in algorithms that require efficient searching.
- **Applicability:** Binary search can only be applied to sorted data. If the data is unsorted, it must be sorted first, which adds an additional $O(n \log n)$ time complexity for sorting, making it less efficient than linear search for small or unsorted datasets.

Task 4: Choosing Between Quick Sort and Merge Sort for Data Processing

Quick Sort:

Prompt:

Complete the recursive implementation of Quick Sort in Python. Add a proper docstring. Explain how recursion works in Quick Sort. Mention best, average, and worst-case time complexity.

Code:

```
def quick_sort(arr):
    """
    Parameters:
    arr (list): The list of elements to be sorted.
    Returns:
    list: A new sorted list containing all elements from the input list.
    """
    if len(arr) <= 1:
        return arr
    else:
        pivot = arr[len(arr) // 2] # Choosing the middle element as pivot
        left = [x for x in arr if x < pivot] # Elements less than pivot
        middle = [x for x in arr if x == pivot] # Elements equal to pivot
        right = [x for x in arr if x > pivot] # Elements greater than pivot
        return quick_sort(left) + middle + quick_sort(right)
# Example usage:
if __name__ == "__main__":
    arr = [3, 6, 8, 10, 1, 2, 1]
    sorted_arr = quick_sort(arr)
    print("Sorted array:", sorted_arr)
```

Output:

```
Sorted array: [1, 1, 2, 3, 6, 8, 10]
```

Explanation:

Sorts an array using the Quick Sort algorithm.

Quick Sort is a divide-and-conquer algorithm that works by selecting a 'pivot' element from the array and partitioning the other elements into two sub-arrays, according to whether they are less than or greater than the pivot. The sub-arrays are then sorted recursively.

The steps of the Quick Sort algorithm are as follows:

- Choose a pivot element from the array.
- Partition the array into two sub-arrays: elements less than the pivot and elements greater than the pivot.
- Recursively apply the above steps to the sub-arrays until they are sorted.

Time Complexity:

- Best Case: $O(n \log n)$ - occurs when the pivot divides the array into two equal halves.
- Average Case: $O(n \log n)$ - occurs on average for random pivots.
- Worst Case: $O(n^2)$ - occurs when the smallest or largest element is always chosen as the pivot (e.g., when the array is already sorted or reverse sorted).

Space Complexity:

$O(\log n)$ on average due to recursive stack space, but can go up to $O(n)$ in the worst case.

Merge Sort:

Prompt:

Complete the recursive implementation of Merge Sort in Python. Add docstring. Explain divide and conquer approach. Mention time complexity.

Code:

```
def merge_sort(arr):  
    if len(arr) > 1:  
        mid = len(arr) // 2 # Finding the mid of the array  
        L = arr[:mid]       # Dividing the elements into 2 halves  
        R = arr[mid:]  
        merge_sort(L)       # Sorting the first half  
        merge_sort(R)       # Sorting the second half  
        i = j = k = 0  
        # Copy data to temp arrays L[] and R[]  
        while i < len(L) and j < len(R):  
            if L[i] < R[j]:  
                arr[k] = L[i]  
                i += 1  
            else:  
                arr[k] = R[j]  
                j += 1  
            k += 1  
        # Checking if any element was left
```

```

        while i < len(L):
            arr[k] = L[i]
            i += 1
            k += 1
        while j < len(R):
            arr[k] = R[j]
            j += 1
            k += 1
# Example usage
if __name__ == "__main__":
    arr = [38, 27, 43, 3, 9, 82, 10]
    print("Original array:", arr)
    merge_sort(arr)
    print("Sorted array:", arr)

```

Output:

```

Original array: [38, 27, 43, 3, 9, 82, 10]
Sorted array: [3, 9, 10, 27, 38, 43, 82]

```

Explanation:

- Sorts an array using the Merge Sort algorithm.
- Merge Sort is a divide and conquer algorithm that divides the input array into two halves, calls itself for the two halves, and then merges the two sorted halves. The merge() function is used for merging two halves. The merge(arr, l, m, r) is a key process that assumes that arr[l..m] and arr[m+1..r] are sorted and merges the two sorted sub-arrays into one.
- **Time Complexity:** $O(n \log n)$ in all cases (best, average, worst) due to the divide and conquer approach.

Comparison:

Quick Sort and Merge Sort are two popular sorting algorithms that have different characteristics and performance based on the type of data being sorted.

1. Random Data:

- **Quick Sort:** Quick Sort is generally faster for random data due to its in-place sorting and lower constant factors. It has an average time complexity of $O(n \log n)$ and is often preferred for random data.
- **Merge Sort:** Merge Sort has a time complexity of $O(n \log n)$ as well, but it requires additional space for the temporary arrays used during the merging process. It may not be as efficient as Quick Sort for random data due to this overhead.
- **Preferred Algorithm:** Quick Sort is preferred for random data due to its faster performance and lower space complexity.

2. Already Sorted Data:

- **Quick Sort:** Quick Sort can perform poorly on already sorted data, especially if the pivot selection is not optimal (e.g., always choosing the first or last element as the pivot). In such cases, it can degrade to $O(n^2)$ time complexity.
- **Merge Sort:** Merge Sort performs well on already sorted data, maintaining its $O(n \log n)$ time complexity. It does not depend on the order of the input data, making it a good choice for already sorted data.
- **Preferred Algorithm:** Merge Sort is preferred for already sorted data due to its consistent $O(n \log n)$ performance regardless of input order.

3. Reverse Sorted Data:

- **Quick Sort:** Similar to already sorted data, Quick Sort can also perform poorly on reverse sorted data if the pivot selection is not optimal. It can again degrade to $O(n^2)$ time complexity.
- **Merge Sort:** Merge Sort will still maintain its $O(n \log n)$ time complexity on reverse sorted data, as it does not rely on the order of the input.
- **Preferred Algorithm:** Merge Sort is preferred for reverse sorted data due to its consistent performance regardless of input order.

Task 5: Optimizing a Duplicate Detection Algorithm

Naive Approach:

Prompt:

Write a brute-force Python algorithm to detect duplicate user IDs using nested loops. Explain the time complexity.

Code:

```
def detect_duplicate_user_ids(user_ids):
    duplicates = []
    for i in range(len(user_ids)):
        for j in range(i + 1, len(user_ids)):
            if user_ids[i] == user_ids[j]:
                duplicates.append(user_ids[i])
                break # Break to avoid adding the same duplicate multiple times
    return duplicates

# Example usage
user_ids = ['user1', 'user2', 'user3', 'user1', 'user4', 'user2']
duplicates = detect_duplicate_user_ids(user_ids)
print("Duplicate user IDs:", duplicates)
```

Output:

```
Duplicate user IDs: ['user1', 'user2']
```

Explanation:

The time complexity of this algorithm is $O(n^2)$ because it uses nested loops to compare each user ID with every other user ID in the list. The outer loop runs n times, and for each iteration of the outer loop, the inner loop also runs up to n times (in the worst case). Therefore, the total number of comparisons made is proportional to $n * n$, resulting in $O(n^2)$ time complexity. This means that as the number of user IDs increases, the

time taken to detect duplicates grows quadratically, making it inefficient for large datasets.

Optimized Approach:

Prompt:

Suggest an optimized duplicate detection algorithm using sets or dictionaries. Rewrite the algorithm. Explain why it improves performance. Compare time complexity with the brute-force method.

Code:

```
def find_duplicates(items):
    seen = set()
    duplicates = set()
    for item in items:
        if item in seen:
            duplicates.add(item)
        else:
            seen.add(item)
    return duplicates

# Example usage:
items = [1, 2, 3, 4, 2, 5, 1]
duplicates = find_duplicates(items)
print(duplicates) # Output: {1, 2}
```

Output:

```
{1, 2}
```

Explanation:

- This algorithm improves performance because sets in Python are implemented as hash tables, which provide average-case $O(1)$ time complexity for membership tests and insertions. In contrast, the brute-force method typically involves nested loops to compare each item with every other item, resulting in $O(n^2)$ time complexity.
- In the optimized algorithm, we only need to iterate through the list once ($O(n)$) and perform $O(1)$ operations for each item, leading to an overall time complexity of $O(n)$. In contrast, the brute-force method requires $O(n^2)$ time due to the nested loops. Therefore, the optimized algorithm is significantly more efficient, especially for larger datasets.

Comparison:

- When comparing $O(n^2)$ and $O(n)$ duplicate detection algorithms for large datasets, such as 1 million records, the difference in performance becomes significant.
- An $O(n^2)$ algorithm would require 1 trillion comparisons ($1 \text{ million} * 1 \text{ million}$) to detect duplicates, which is computationally expensive and time-consuming. In contrast, an $O(n)$ algorithm would only require 1 million comparisons, making it much more efficient and faster.
- Optimization is crucial in this context because it can significantly reduce the time and resources required to process large datasets. With an $O(n)$ algorithm, you can quickly identify duplicates without the need for extensive computational power, which is essential when dealing with large volumes of data. This efficiency can lead to faster decision-making, improved user experience, and reduced costs associated with processing and storage. Therefore, optimizing algorithms for duplicate detection is vital to ensure scalability and performance in real-world applications.